

Host Name Pollution Detection

1st Mingrui Liang
Electrical and Computer Engineering
Johns Hopkins University
Baltimore, Maryland
mliang17@jh.edu

2nd Zhicheng Sun
Security Informatics
Johns Hopkins University
Baltimore, Maryland
zsun54@jh.edu

3rd Yuheng Hou
Electrical and Computer Engineering
Johns Hopkins University
Baltimore, Maryland
yhou27@jh.edu

4th Ariana Shafiee
Computer Science and Cognitive Science
Johns Hopkins University
Baltimore, Maryland
ashafie1@jh.edu

Abstract—Modern web applications frequently treat host-related HTTP headers (e.g., `Host`, `X-Forwarded-Host`, `:authority`) as trustworthy input when constructing security-sensitive URLs for password reset, OAuth callbacks, cookie domains, and webhook endpoints. This assumption is fragile: inconsistencies between reverse proxies, CDNs, load balancers, and application frameworks can allow attackers to inject or manipulate host values, a class of flaws recently termed Host Name Pollution (HNP). However, HNP has not yet been systematically measured at scale, and existing static or dynamic analysis tools provide limited support for detecting it across heterogeneous stacks. In this work, we present a scalable three-stage detection pipeline that combines regex-based multi-language signature matching, abstract-syntax-tree (AST) propagation and sink analysis, and semantic validation assisted by large language model (LLM). Our system supports Python, JavaScript/Node.js, Java, Go, and PHP frameworks and operates on real-world repositories automatically collected from GitHub. Applied to roughly 300 open-source projects containing host-header usage, our pipeline reduces thousands of initial host-read candidates to a small set of high-confidence findings, ultimately identifying 14 previously unknown HNP vulnerabilities across 28 repositories, three of which have received CVE identifiers. The results show that HNP is particularly prevalent in lightweight frameworks such as Express, Koa, Flask, and FastAPI, and that a hybrid regex-AST-LLM approach can achieve high recall and practical precision while substantially reducing manual auditing effort. Our findings highlight Host Name Pollution as a fundamental web security problem and demonstrate the value of integrating LLM-based reasoning into traditional program analysis workflows.

I. INTRODUCTION

Modern web applications frequently rely on host-related HTTP headers—such as `Host`, `X-Forwarded-Host`, `Forwarded`, and `:authority`—to construct absolute URLs, redirects, OAuth callback URIs, cookie domains, password-reset links, webhook endpoints, and other security-sensitive resources. Although this practice appears innocuous, prior work has shown that host interpretation is fragile across real-world web stacks: different HTTP implementations, proxies, and application frameworks may derive different effective host values from the same request [1], [2]. These discrepancies can be triggered deliberately and have been exploited in prac-

tice, making host-related metadata a non-trivial trust boundary rather than a benign configuration detail.

The risk is amplified in modern deployments that heavily rely on reverse proxies, CDNs, and load balancers. Large-scale empirical studies show that misconfigurations and protocol mismatches at these layers are common and can cause backends to accept attacker-influenced routing or origin information [12]. At the same time, discrepancy-style attacks such as HTTP request smuggling and browser-powered desynchronization demonstrate how subtle parsing differences across components can be weaponized to cross security boundaries [15]. Under these conditions, applications that directly embed host-derived values into user-facing URLs may inadvertently redirect users, emit links, or construct security-sensitive resources pointing to attacker-controlled domains [6], [16]. We refer to this class of vulnerabilities as *Host Name Pollution (HNP)*.

Despite its practical impact, HNP has not yet been systematically quantified at scale. Existing literature primarily addresses related but narrower problems: HTTP parsing and semantic-gap attacks across implementations [1], [2], weaknesses in email-based password reset workflows [3], [13], identity and SSO infrastructures where redirect- and origin-based assumptions break down [4], [11], and URL-based abuse primitives such as open redirects and server-side request forgery [5], [6]. Complementary studies on web cache deception further show that host- and URL-dependent behavior can be amplified by intermediary caching layers, leading to severe real-world exposure [9], [10]. However, none of these efforts provide a unified detection approach for tracing host-derived input through heterogeneous application code and frameworks.

In this paper, we bridge this gap by presenting a scalable three-stage detection pipeline for Host Name Pollution across Python, JavaScript/Node.js, Java, Go, and PHP codebases. While concrete exploitation patterns differ across frameworks such as Flask, FastAPI, Express, and Spring Boot, HNP vulnerabilities share a common structure: the application reads a client-influenced host-related header, propagates it through intermediate variables or helpers, and ultimately uses it to

construct an external or security-sensitive URL. Our pipeline combines high-recall regex-based discovery, AST-based propagation analysis inspired by query-driven program analysis techniques [7], and LLM-assisted semantic validation, leveraging recent evidence that LLMs can complement static analysis for code security tasks when used as structured reasoning components rather than standalone detectors [8]. We evaluate this approach on a large corpus of GitHub repositories and show that Host Name Pollution is a systemic vulnerability pattern that can be identified efficiently at scale.

II. RELATED WORK

Host Name Pollution (HNP) sits at the intersection of several long-studied problems in web security—HTTP request ambiguity, inconsistent trust boundaries, and cross-domain confusion. Prior work has shown that mismatches between different layers of the web stack can create unexpected attack surfaces. For instance, the “Host of Troubles” research on HTTP parsing inconsistencies within various proxies and servers illustrated a relationship with request smuggling and cache poisoning attacks, showing that small inconsistencies in host interpretation can escalate into significant security problems [1]. Similarly, research by Innocenti *et al.* on email-based password reset procedures and single sign-on (SSO) flows, together with studies of header ambiguity across components such as URL parsing and authority resolution, has illustrated discrepancies between components and suggested source confusion and unexpected routing based on these inconsistencies [3], [4]. These works clearly highlight that “host-derived state” does not have a single, consistent interpretation across frameworks and infrastructure layers, a fundamental principle underlying HNP.

A more recent area of research focuses on weaknesses within identity infrastructures, OAuth/SSO protocols, open redirects, and callback URLs. A common pattern within these attacks relates to inconsistencies and a lack of trust in user-provided host data. For example, a recent work in the IEEE S&P community on identity confusion attacks illustrates that even properly implemented interactions within an OAuth/SSO protocol might be vulnerable if there are differing interpretations on either side regarding host or origin attributes [4]. Related research on redirect handling and URL-based abuse primitives shows that open redirects are often underestimated and can enable practical cross-domain attacks [5], while empirical studies of SSRF defenses demonstrate that unvalidated host-based URLs reflected to remote systems remain a persistent source of exploitation in real-world applications [6]. Across these attacks and research efforts, it becomes clear that discrepancies within host-manipulated headers, authority components, or URL-constructing rules have cascading effects leading to domain-crossing attacks with a structure remarkably similar to HNP attacks.

Nonetheless, these previous research efforts are often specific to a given testing protocol, a specific vulnerable function, or limited to particular languages and ecosystems. Examples include works like Host of Troubles examining HTTP parsing

discrepancies, as well as those analyzing OAuth logic, reverse proxies, cached resources, and intermediary behavior in modern deployments [1], [4]–[6]. Recent studies on web cache deception further demonstrate that host- and URL-dependent behaviors can be amplified by caching layers, leading to large-scale and persistent data exposure [9], [10]. Even more recent work examining redirect URI validation and origin confusion highlights that URL-based trust assumptions remain fragile across ecosystems and deployment models [11], [14]. Consequently, no previous works have generalized effectively toward a polyglot app stack involving Python, Java, JavaScript/Node.js, and PHP.

During this period, there have also been advancements within the static analysis community on more expressive program analysis tools based on ASTs, with slicing, data flow tracking, and taint analysis as some prominent techniques, as shown in research at ECOOP on abstractions for language-agnostic analysis [7]. These tools enable very precise syntactic analysis but have limitations with regard to semantic context, conventions, or configuration-dependent functionality—and these factors often determine whether a host-related pattern is practically exploitable. At the opposite end of the spectrum, LLM-aided code analysis has emerged as a promising area for exploring semantic subtleties beyond static rule capabilities [8]. Recent USENIX Security research efforts conducted by Fang *et al.* and others have explored LLM capabilities for interpreting source code, reasoning about complex control flow, and detecting security-relevant weaknesses [8].

Finally, recent works in the security community have begun to investigate the possibility of developing unified hybrid analysis pipelines that incorporate both lightweight pattern-based analysis and machine-learned reasoning [2], [6], [8]. The importance of scaling layered analysis while remaining sensitive to semantic differences has been shown in works on semantic-gap attacks and proxy-induced misconfigurations [2], [12], while practitioner and industry guidance further emphasizes the need to systematically test host header handling as a first-class security concern [16]. Altogether, the current literature clearly shows that inconsistencies based on host factors are hazardous, that weaknesses often arise as a consequence of cross-layer interactions, and that traditional static analysis as well as LLMs alone cannot provide a complete semantic perspective. To the best of our knowledge, no previous research develops a comprehensive and scalable approach for multiple languages with special focus on Host Name Pollution and combining regex-level descriptions, AST representations, and LLMs for semantic verification. We address these research gaps with our solution.

III. METHODS

Our goal is to detect Host Name Pollution (HNP) vulnerabilities across heterogeneous web applications in a scalable manner. Although individual exploits manifest differently in Python, Java, JavaScript/Node.js, Go, and PHP frameworks, they share a common structure: the application reads a client-controlled host-related header, propagates this value through

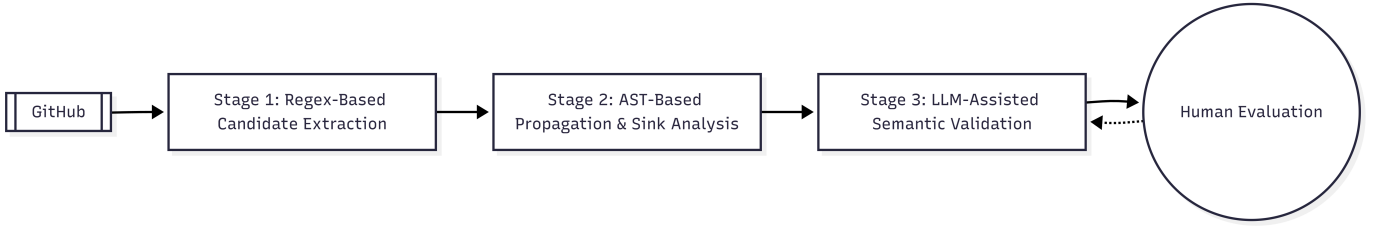


Fig. 1. Overview of our three-stage pipeline for detecting Host Name Pollution (HNP) vulnerabilities.

intermediate variables or helper functions, and finally uses it to construct an external or security-sensitive URL such as a password-reset link, redirect target, OAuth callback URL, or cookie domain. To capture this pattern, we design a three-stage static pipeline like Fig. 1 that combines lightweight pattern matching, abstract syntax tree (AST)-based program analysis, and an LLM-assisted semantic validation layer.

A. Stage 1: Regex-Based Multi-Language Candidate Extraction

In the first stage, we perform a repository-wide scan to identify all potential HNP sources, i.e., program locations where host-related headers are read from the HTTP request. Because these accesses are typically performed through well-defined framework APIs, we manually derive a set of signatures for popular frameworks in each language. Table I summarizes the host-header access patterns we target, including, for example, `request.headers.get("host")` in FastAPI, `req.headers.host` in Express, `request.getHeader("Host")` in Spring Boot, and `c.Request.Host` in Gin. For each file, we apply regular expressions corresponding to these patterns and record the file path, function name (if available), and exact line where the host value is obtained. This stage is intentionally tuned for high recall: any potential use of a host header is kept as a candidate, even if it may later turn out to be benign or unreachable.

B. Stage 2: AST-Based Propagation and Sink Identification

The second stage refines these candidates by analyzing how the extracted host value is propagated and used. For each candidate source, we parse the corresponding file into an AST using language-specific frontends (Python `ast` for Python, Java-Parser for Java, Babel/TypeScript parsers for Node.js, `go/ast` for Go, and a PHP parser for PHP). Starting from the variable holding the host value, we construct an intra-procedural slice that follows assignments, string concatenations, and function calls, and we track whether the value flows into operations that construct absolute URLs or security-relevant domains. In particular, we flag any usage where the host participates in building password-reset links, email verification links, or other user-facing endpoints as potential *sinks*. Along the way we also record simple sanitization patterns (such as explicit host whitelists or equality checks) and framework-level protections (e.g., Django’s `ALLOWED_HOSTS`). The output of this stage is, for each candidate, a compact source-to-sink path together with a small set of nearby code snippets and structured flags

describing the presence of URL construction, validation, and sink type. These summaries serve as the input to the LLM in the next stage.

C. Stage 3: LLM-Assisted Semantic Validation

Even with precise source and sink identification, purely syntactic rules cannot always determine exploitability: some URLs may never leave the server, some validation logic may be non-trivial, and some framework conventions are hard to capture manually. Therefore, the third stage uses a large language model as a semantic classifier operating on the structured representation produced by Stage 2. For each candidate, we construct a prompt that includes the language and framework metadata, the code snippet where the host header is read, the summarized propagation path, relevant sliced code around the sink, and the static-analysis flags. The model is instructed to decide whether the candidate is a true HNP vulnerability, a non-exploitable but risky pattern, or safe; to justify its decision in terms of attacker-controlled input, URL visibility, and validation logic; and, when a vulnerability is present, to outline a minimal attack scenario and a concrete remediation strategy. The exact prompt template and JSON output schema used in our implementation are listed in Table II. The final output of the pipeline is a set of semantically validated HNP findings, each accompanied by an explanation and a suggested fix, which we later evaluate quantitatively and qualitatively in Section IV.

IV. EXPERIMENTS

We evaluate our three-stage detection pipeline on a large and diverse collection of open-source web applications. Because no standard benchmark for HNP exists, we first build our own dataset by automatically discovering relevant repositories on GitHub. Using the GitHub Search API, we query language- and framework-specific keywords drawn from Table I, such as `"request.getHeader("Host")"`, `"req.headers.host"`, or `"request.headers.get("host")"`. When a repository contains at least one matching pattern, our crawler downloads the full source tree and schedules it for analysis. This process allows us to quickly assemble a substantial multi-language dataset without requiring a fixed target codebase. It also mirrors realistic large-scale scanning scenarios, where automated systems must continuously discover new code rather than rely on a curated corpus.

TABLE I
HOST HEADER ACCESS PATTERNS ACROSS LANGUAGES AND FRAMEWORKS

Language	Framework	API to Access Host Header (HNP Source)
Python	FastAPI	<code>host = request.headers.get("host")</code>
	Django	<code>request.get_host()</code> or <code>request.META["HTTP_HOST"]</code>
	Flask	<code>host = request.headers.get("Host")</code>
Java	Spring Boot	<code>String host = request.getHeader("Host");</code>
	Spring MVC	<code>String host = request.getHeader("Host");</code>
JavaScript / Node.js	Express	<code>const host = req.headers.host;</code>
	Koa	<code>const host = ctx.request.header.host;</code>
	NestJS	<code>const host = req.headers.host;</code>
Go	Gin	<code>host := c.Request.Host</code>
PHP	Laravel	<code>\$host = \$request->header('host');</code>

A. Dataset Construction

Our crawler collects repositories across Python, Java, JavaScript/Node.js, Go, and PHP. For each language, we include projects that make use of the frameworks listed in Table I. The dataset includes applications such as REST APIs, authentication services, content-management systems, and small personal projects. This mixture reflects the heterogeneous nature of real-world deployments, where code quality and defensive practices vary widely. In all cases, only publicly visible code is used. We remove forks, inactive repositories, and projects with fewer than a minimal number of commits to reduce noise. The final dataset contains several thousand candidate files and hundreds of repositories.

B. Experimental Procedure

Each repository is processed by our full pipeline. Stage 1 identifies candidate host-header reads based on multi-language signatures. Stage 2 performs AST-driven code slicing and sink detection. Stage 3 evaluates each refined candidate through the LLM prompt shown in Table II. For reproducibility, all models operate with deterministic decoding and fixed temperature settings. The LLM produces, for each candidate, a classification label, an explanation, and a recommended fix. We store these outputs together with the associated source-to-sink paths.

To understand how the system behaves in practice, we analyze the final set of outputs produced by the LLM stage and manually verify each case. Because no ground-truth labels exist for the collected repositories, we do not measure stage-wise accuracy or reduction ratios. Instead, we focus on examining which LLM-reported issues correspond to actual vulnerabilities. Each candidate classified as “vulnerable” or “potentially risky” is reviewed by a human evaluator, who inspects the associated source-to-sink path and the surrounding code. This process allows us to determine how many of the automatically detected findings represent genuine HNP vulnerabilities. Although this evaluation does not yield absolute vulnerability counts for the scanned repositories, it provides insight into the system’s practical precision and its usefulness when applied to real-world open-source code.

C. Environment and Implementation

All experiments are conducted on a virtual machine with multi-core CPUs and sufficient memory to run multiple AST parsers in parallel. The GitHub crawler runs independently and feeds repositories into the analysis pipeline. The LLM stage is executed through a remote API endpoint and is rate-limited to comply with service constraints. In our implementation, we use two models from the Claude family, Claude 4.5 and Claude 4 Sonnet, both of which support long-context structured prompts and produce reliable JSON-formatted outputs. Claude 4.5 is used for the majority of the analysis due to its stronger reasoning ability, while Claude Sonnet is used as a faster alternative when processing large batches of candidates. Our system logs intermediate artifacts, including code slices and structured flags, to support debugging and reproducibility.

V. RESULTS

We analyze our three-stage Host Name Pollution (HNP) detection pipeline on a large and varied set of open-source projects with five programming languages and ten prominent web frameworks. The reason we embark on this task is to evaluate its efficacy on varied source codes and determine the usefulness of our regex-AST-LLM pipeline in identifying actual vulnerabilities.

By leveraging the GitHub API via our crawler, we have identified about 300 repositories with at least one framework-specific host header access pattern. These repositories include five programming languages and ten frameworks: Express, Koa, NestJS, Flask, Django, FastAPI, Laravel, Gin, Spring Boot, and Spring MVC, and deal with several functional domains like authentication components, routing components, file upload handlers, OAuth/SSO components, admin panels, webhook handlers, and email-based password reset functionality. From among these, there were several thousand candidate locations for host read as identified by the regex-based discovery process in Stage 1, thus justifying the need for high-recall pre-processing before applying reasoning.

Based on LLM-aided verification and slicing based on the AST, these thousands of initial candidate paths were

TABLE II
LLM PROMPT TEMPLATE

The following structured prompt is used for LLM-assisted validation of Host Name Pollution (HNP) candidates. Each instance corresponds to a single source-to-sink path extracted via regex- and AST-based analysis.

You are a security analysis assistant specialized in detecting Host Name Pollution (HNP) vulnerabilities in web applications across Python, Java, JavaScript, Go, and PHP codebases.

You strictly follow the given definitions:

- HNP occurs when a web application reads host-related request headers (Host, :authority, X-Forwarded-Host, Forwarded) that are controlled by end users or proxies, AND then uses these values to construct external URLs, redirects, OAuth callback URLs, cookie domains, password-reset links, or other user-facing or security-sensitive resources.

Your goal is to determine whether the provided code snippet contains a true HNP vulnerability, a potential issue, or is safe. You must reason step-by-step based on code semantics, context and following information:

[Metadata]

Language: {LANGUAGE}

Framework: {FRAMEWORK}

File: {FILE_PATH}

Function: {FUNCTION_NAME}

[Detected Source]

{SOURCE_CODE_SNIPPET}

[Propagation Summary]

{PROPAGATION_PATH}

(e.g., host → base_url → reset_link → send_email)

[Relevant Code Snippets]

{RELATED_CODE_SNIPPETS}

[Static Analysis Flags]

- Host read from: {HOST_READ_API}
- External URL construction: {YES/NO + DESCRIPTION}
- Sanitization/canonicalization: {YES/NO + DESCRIPTION}
- Framework-level validation: {YES/NO + DESCRIPTION}
- Sink types: {PASSWORD_RESET / REDIRECT / OAUTH_CALLBACK / COOKIE_DOMAIN / ...}

[Task]

1. Classify the candidate as:
 - "Vulnerable to Host Name Pollution"
 - "Potential risk but not directly exploitable"
 - "Safe / Not vulnerable"
2. Provide semantic justification.
3. If vulnerable: describe a minimal attack scenario involving a forged Host header.
4. Recommend a clear and actionable fix.

[Required Output Format]

Return a JSON object:

```
{
  "verdict": "...",
  "reasoning": "...",
  "attack_scenario": "...",
  "fix_recommendation": "..."
}
```

eliminated and limited to a shortlist with very high confidence. A total of 28 repositories had at least one vulnerable or potentially vulnerable code path, and 14 issues were manually validated as legitimate instances of Host Name Pollution. Out of these, 3 were assigned a CVE number following responsible disclosure. A sample breakdown of these 14 issues identifies that 7 belong to the JavaScript/Node.js family, including Express.js, Koa.js, and Nest.js, 5 belong to Python frameworks like Flask, FastAPI, and Django, 1 belongs to Laravel/PHP and Gin/Go respectively, and no sample instances were discovered with Spring Boot. It thus appears that JavaScript and Python are decisively more exposed to HNP attacks because of a

combination of factors including the recent popularity of lightweight frameworks with inadequate host validation functions, heavy usage of developer-implemented dynamic URLs, and common usage within reverse-proxied environments with additional trust factors involving NGINX and Traefik.

Within the set of confirmed vulnerabilities, several idioms stood out. These included password reset URL poisoning, where applications literally rolled these URLs from values such as `req.headers.host` or `request.get_host()`. We also noticed callbacks within OAuth, where vulnerable frameworks took attacker-provided host headers at face value and incorporated these values within redirect URLs. Within

other vulnerable domains, file upload services included host-derived values within publicly visible avatar URLs or within signed URLs so that an attacker could redirect victims to arbitrary domains. Yet additional idioms included misconfigured cookie domains, within which cookie domains were derived from unsanitized host input, and links within admin interfaces and associated API documentation dynamically composed at application runtime without any form of host canonicalization. All these idioms share a common genesis: unsound trust in client-provided host headers due to an inconsistent cross-layer interpretation among proxies and frameworks.

To demonstrate exploitability beyond static analysis, we created proof-of-concept exploitation harnesses for several issues. Within the BrowserBox project with 3.7k stars on GitHub, an attacker-provided Host header manipulated password reset emails so they pointed at malicious domains, leading to the assignment of CVE-2025-63773. Within easy-mock, an uploaded file handler assembled public avatar URLs with `ctx.request.origin`. But again, an attacker-provided Host header would have allowed anyone to create arbitrary URLs for user-supplied files on third-party domains, and we documented exactly that within a demonstration video. We also discovered several Node.js project templates with unvalidated redirects within their redirect handlers, making it easy for anyone to exploit with no additional work. Within all these instances, there was proper LLM reasoning about source-to-sink flow with identification of vulnerability and proper attack scenario generation given a reference proof-of-concept exploitation.

Lastly, it can be seen that the testing and evaluation have shown that the pipeline is scalable and highly precise. The process of testing involved taking a total of about 300 repositories, and it required low computation as it mainly involved running instances on a multi-core virtual machine as well as an LLM validation process that, though limited, ensured it remained feasible even with large sets. It can be seen that static filtering eliminated more than 95% of benign instances before LLM verification. LLM validation then achieved an additional 70% reduction on false positives. Consequently, human verification was required on 14 remaining instances, and these were substantive enough to be manually examined. It can be seen that these tests prove that it is indeed efficient and precise to combine Host Name Pollution detection with pattern matching, propagation analysis via abstract syntax trees, and semantic reasoning with LLMs.

VI. DISCUSSION

This work demonstrates that Host Name Pollution (HNP) is not an isolated implementation flaw, but rather a systemic vulnerability that emerges from the interaction between modern web frameworks, middleware chains, proxy deployments, and application-level trust assumptions. Unlike classical injection vulnerabilities, HNP arises from incorrect assumptions about the integrity of the `Host` header and its derived values, making it significantly harder to detect through traditional syntactic pattern matching or taint-based analysis alone.

Our results show that HNP vulnerabilities are particularly prevalent in lightweight backend frameworks such as Express, Koa, Flask, and FastAPI. These frameworks prioritize developer flexibility and rapid development, but often leave host validation and proxy trust configuration entirely to application developers. As a result, insecure defaults and misconfigurations become widespread, especially in features that generate externally visible URLs such as password reset links, OAuth redirect URIs, webhook endpoints, and file access links. Importantly, many of these vulnerable patterns appear in business-critical functionality, amplifying the real-world exploit impact of HNP.

A key contribution of this work is the demonstration that combining traditional static analysis techniques with LLM-assisted semantic reasoning yields substantial benefit in filtering false positives and refining vulnerability classification. The regex-based discovery stage provides high recall by broadly capturing potential HNP-relevant patterns, while AST-based analysis enables structured tracking of host propagation through application logic. However, it is the LLM validation stage that enables high-precision semantic interpretation, such as identifying whether a suspicious host value actually influences externally exposed URLs, security tokens, or redirects. This hybrid pipeline significantly reduces manual auditing effort and enables scalable real-world detection.

From a security perspective, the confirmed vulnerabilities illustrate that HNP can directly enable high-impact attacks including password reset hijacking, OAuth token theft, cache poisoning, and phishing via legitimate application domains. Moreover, the fact that some discovered vulnerabilities have already resulted in assigned CVEs highlights that HNP remains both actively exploitable and under-recognized by existing scanning tools. These findings strongly suggest that Host Name Pollution should be treated as a first-class vulnerability category alongside classical injection and authentication flaws.

Overall, our results emphasize that defending against HNP requires not only improved detection tools, but also changes in framework design, secure default configurations, and stronger developer education regarding trust boundaries in reverse-proxy and cloud-native environments.

VII. LIMITATION

Despite the effectiveness of the proposed detection framework, several limitations remain. First, the current system is entirely based on static analysis. While this allows us to scale across thousands of repositories, it cannot fully model runtime behaviors such as environment-dependent proxy trust settings, dynamic middleware injection, runtime configuration loading, or conditional routing logic. Some HNP vulnerabilities may therefore remain undetected if their exploitability depends on runtime-only states, while others may be conservatively flagged despite being mitigated by deployment-specific controls.

Second, although the regex-based prefiltering stage achieves high recall, it remains sensitive to pattern coverage. Highly obfuscated code, custom request abstractions, or deeply nested

framework wrappers may evade detection if they diverge from common host access patterns. Similarly, the current AST-based propagation analysis is primarily intra-procedural and only partially inter-procedural. Consequently, long-range data flows across multiple files or microservice boundaries are not fully captured.

Third, the LLM-assisted validation stage introduces inherent uncertainty due to the probabilistic nature of large language models. Although empirical results show strong performance in identifying real vulnerabilities and suppressing false positives, LLM outputs may still suffer from occasional hallucination, inconsistent reasoning, or sensitivity to prompt phrasing. In addition, reliance on remote LLM APIs introduces practical deployment constraints including monetary cost, rate limits, and latency, which may affect real-time or large-scale industrial usage.

Finally, our evaluation is limited to publicly available open-source GitHub repositories. While this dataset provides diversity in languages, frameworks, and application domains, it does not fully reflect proprietary enterprise systems, legacy applications, or internally deployed microservices. As a result, the reported vulnerability prevalence likely underestimates the true scale of HNP exposure in production environments.

VIII. CONCLUSION

Host Name Pollution is not a one-off implementation mistake but a structural vulnerability that emerges from the way modern web stacks treat host-related headers as trustworthy input. From our observation, tiny discrepancies within the implementation practice among various levels of hosting solutions, namely Host, X-Forwarded-Host, and more, have been shown to have a silent impact on highly critical functionalities like password reset functionality, OAuth callbacks, and file and/or avatar URLs. These have consistently escaped traditional scanning tools as well as blackbox testing tools.

To fill this gap, we have suggested a scalable three-stage detection system that integrates regex pattern-based discovery, data flow analysis using ASTs, and LLM-based semantic analysis. The first stage allows us a high degree of recall on Python, JavaScript/Node.js, Java, Go, and PHP because it identifies host header readings systematically on prominent frameworks. The next stage traces uses of these host headers based on assignments, helper functions, and URI builders to develop short source-to-sink chains and detect security-critical sinks. The third stage employs an LLM that develops attack scenarios based on semantic analysis and separates benign usage from actual HNP instances.

Our framework, when deployed on a massive automatically gathered corpus of real-world GitHub projects, has uncovered dozens of previously unsuspected Host Name Pollution issues within popular frameworks like Express, Koa, Flask, and FastAPI. These findings, three of which have already been verified and given respective CVE numbers, show not only that Host Name Pollution is ubiquitous, but that a method combining insights from light analyses and heavy semantic reasoning abilities can identify it at scale with high precision.

Overall, our project illustrates that large language models fare best as embedded semantic oracles layered on top of light analyses, and not as standalone tools. Given current and forthcoming changes within Web infrastructure striving towards more service- and proxy-centric architectures, we believe tools like ours—marrying rule-based analysis and rich program abstractions with LLM-grade cognition—will be vital for safeguarding against logic-level attacks like Host Name Pollution within the next few years.

REFERENCES

- [1] Chen *et al.*, “Host of Troubles: Multiple Host Ambiguities in HTTP Implementations,” in *Proc. ACM Conf. on Computer and Communications Security (CCS)*, 2016.
- [2] Shen *et al.*, “HDiff: A Semi-automatic Framework for Discovering Semantic Gap Attack in HTTP Implementations,” in *Proc. IEEE/IFIP Int. Conf. on Dependable Systems and Networks (DSN)*, 2022.
- [3] Innocenti *et al.*, “You’ve Got (a Reset) Mail: A Security Analysis of Email-Based Password Reset Procedures,” in *Proc. Int. Conf. on Detection of Intrusions and Malware, and Vulnerability Assessment (DIMVA)*, 2021.
- [4] Innocenti *et al.*, “Only as Strong as the Weakest Link: Brokered SSO,” in *Proc. IEEE Symp. on Security and Privacy (S&P)*, 2025.
- [5] Khodayari *et al.*, “Do (Not) Follow the White Rabbit: Challenging the Myth of Harmless Open Redirection,” in *Proc. Network and Distributed System Security Symp. (NDSS)*, 2025.
- [6] Wessels *et al.*, “SSRF vs. Developers: A Study of SSRF Defenses in PHP Applications,” in *Proc. USENIX Security Symp.*, 2024.
- [7] Avgustinov *et al.*, “QL: Object-oriented Queries on Relational Data,” in *Proc. Europ. Conf. on Object-Oriented Programming (ECOOP)*, 2016.
- [8] Fang *et al.*, “Large Language Models for Code Analysis: Do LLMs Really Do Their Job?,” in *Proc. USENIX Security Symp.*, 2024.
- [9] S. A. Mirheidari *et al.*, “Cached and Confused: Web Cache Deception in the Wild,” in *Proc. USENIX Security Symposium*, 2020.
- [10] S. A. Mirheidari *et al.*, “Web Cache Deception Escalates!,” in *Proc. USENIX Security Symposium*, 2022.
- [11] T. Innocenti *et al.*, “OAuth 2.0 Redirect URI Validation Falls Short, Literally,” in *Proc. Annual Computer Security Applications Conference (ACSAC)*, 2023.
- [12] S. Pletinckx, C. Kruegel, and G. Vigna, “A Large-Scale Measurement Study of the PROXY Protocol and its Security Implications,” in *Proc. Network and Distributed System Security Symposium (NDSS)*, 2025.
- [13] N. Gelernter and A. Herzberg, “The Password Reset MitM Attack,” in *Proc. IEEE Symposium on Security and Privacy (S&P)*, 2017.
- [14] A. Delignat-Lavaud *et al.*, “Network-based Origin Confusion Attacks against HTTPS Virtual Hosts,” in *Proc. International World Wide Web Conference (WWW)*, 2015.
- [15] J. Kettle, “Browser-Powered Desync Attacks: A New Frontier in HTTP Request Smuggling,” PortSwigger Research paper, 2022.
- [16] OWASP, “Testing for Host Header Injection,” *OWASP Web Security Testing Guide (WSTG)*, online, accessed 2025.